

CƠ CHẾ NGẮT (INTERRUPT), VECTOR TABLE

1. Nhắc lại về Con trỏ hàm (Function Pointer)

Trước khi đi sâu vào cơ chế ngắt, chúng ta cần hiểu cách C/C++ quản lý địa chỉ của một hàm:

```
#include <stdio.h>

void function(int x)
{
    printf("This is the function \\n");
}

int main()
{
    function(1);                // Execute the function
    // 'function' without parentheses represents the function's memory address
    void (*pf)(int) = function; // Assign function address to pointer 'pf'

    return 0;
}
```

Kiến thức lập trình C/C++ cần phải nắm:

- `function(1)` : Khi ta gọi tên hàm và **có** truyền input vào (có dấu ngoặc tròn), hàm đó sẽ được thực thi.
- `function` : Khi ta gọi tên hàm mà **KHÔNG** truyền input vào (không có dấu ngoặc tròn), nó sẽ đại diện và trả về **địa chỉ của hàm** đó trong bộ nhớ.

2. Luồng thực thi khi có ngắt (Interrupt) xảy ra

Khi có một sự kiện ngắt (ví dụ: nhấn nút gửi tín hiệu EXTI0), vi điều khiển sẽ xử lý theo các bước sau:

1. Thiết bị ngoại vi (Peripheral) gửi tín hiệu ngắt (Interrupt signal) lên bộ điều khiển ngắt trung tâm **NVIC**.
2. CPU tạm dừng hàm `main()`.
3. **NVIC** tra cứu trong bảng **Vector Table** để tìm địa chỉ của ngắt EXTI0 (nằm ở offset `0x00000058`).
4. Tại vùng nhớ `0x00000058`, NVIC đọc ra được một giá trị (ví dụ: `0x0800035D`). Đây chính là địa chỉ của hàm thực thi ngắt (`EXTI0_IRQHandler`).

- Bằng cách chạy debug, chúng ta có thể đọc được giá trị ở địa chỉ `0x58` đang chứa `0x0800035D` (Lưu ý: *firmware khác nhau có thể giá trị ở 0x58 sẽ khác nhau*).
- Bằng cách xem file `.map`, chúng ta có thể thấy `EXTI0_IRQHandler` đang nằm ở `0x0800035C`.

5. CPU nhảy tới địa chỉ này để thực thi hàm `EXTI0_IRQHandler`.

Lưu ý về tập lệnh Thumb của ARM Cortex-M: Khi debug vào bộ nhớ của STM32, nếu xem file `.map`, bạn sẽ thấy `EXTI0_IRQHandler` được lưu ở `0x0800035C`. Tuy nhiên, giá trị lưu trong Vector Table lại là `0x0800035D`. Tại sao có sự chênh lệch 1 byte? Phép toán: `0x0800035C | 1 = 0x0800035D`. Bit LSB bằng 1 để báo cho core ARM biết rằng nó cần thực thi tập lệnh ở chế độ **Thumb state** (chế độ tập lệnh 16-bit/32-bit của Cortex-M).

3. Vấn đề: Thay đổi hàm phục vụ ngắt lúc Runtime

Giả sử chúng ta không muốn nhảy vào hàm mặc định `EXTI0_IRQHandler` mà muốn trỏ tới một hàm tự định nghĩa là `Button_Handler`:

```
void Button_Handler()
{
    // Custom logic for handling button press
}
```

Theo logic thông thường, ta chỉ cần ghi đè địa chỉ của `Button_Handler` vào vị trí của EXTI0 trong Vector Table:

```
uint32_t* px = (uint32_t*)0x58;
*px = (uint32_t)Button_Handler; // Assign new function ad
```

dress to Vector Table

Nhưng XUI THAY, cách này sẽ không thành công!

Nguyên nhân: Địa chỉ `0x58` nằm trong dải `0x00000000 - 0x0007FFFF`, vùng này được map trực tiếp từ `0x08000000` (Vùng nhớ **FLASH**). Do đặc trưng phần cứng, Flash là vùng nhớ **READ-ONLY** trong quá trình runtime thông thường, ta không thể dùng con trỏ để gán giá trị mới vào đây được.

4. Giải pháp: Dời Vector Table lên RAM (Relocate Vector Table)

Quan sát Memory Map, ta thấy vùng nhớ **SRAM** (`0x20000000 - 0x20020000` đối với STM32) là vùng nhớ hỗ trợ cả **READ** và **WRITE**. Giải pháp ở đây là copy toàn bộ Vector Table từ Flash dời lên RAM.

Bước 4.1: Copy Vector Table từ Flash sang RAM

Giả sử Vector Table có kích thước `0x198` bytes:

```
uint8_t* ram_addr = (uint8_t*)0x20000000;
uint8_t* flash_addr = (uint8_t*)0x08000000; // Standard base Flash address

// Copy Vector Table to RAM byte-by-byte
for(int i = 0; i < 0x198; i++)
{
    *(ram_addr + i) = *(flash_addr + i);
}
```

Bước 4.2: Cập nhật thanh ghi VTOR (Vector Table Offset Register)

Sau khi copy, ta phải báo cho ARM Cortex-M biết rằng Vector Table đã bị dời đi, để khi có ngắt, NVIC sẽ lên RAM tìm địa chỉ hàm thực thi thay vì tìm ở Flash như cũ.

Thanh ghi quản lý việc này là **VTOR** (nằm ở địa chỉ `0xE000ED08` của System Control Block). (đọc tài liệu: *[ref manual] ARM M4.pdf - chapter: 4.3 System Control Block*)

```
uint32_t* VTOR = (uint32_t*)0xE000ED08; // Pointer to VTOR register
```

```
*VTOR = 0x20000000; // Write new Vector  
r Table base address
```

Lúc trước: NVIC tìm địa chỉ hàm thực thi ngắt EXTI0 tại `0x58`.

Bây giờ: NVIC sẽ tìm địa chỉ hàm thực thi ngắt EXTI0 tại `0x20000058`.

Bước 4.3: Ghi đè địa chỉ hàm ngắt mới

Giờ đây, Vector Table đã nằm trên RAM (có quyền WRITE), ta có thể thoải mái thay đổi hàm ngắt:

```
uint32_t* px = (uint32_t*)0x20000058; // EXTI0 offset on  
RAM Vector Table  
*px = (uint32_t)Button_Handler; // Overwrite with  
Button_Handler address
```

5. GIẢI QUYẾT BÀI TẬP VỀ NHÀ (HOMEWORK)

Yêu cầu 1: Làm lại luồng nhảy tới `Button_Handler` khi EXTI0 gửi interrupt

Yêu cầu 2: Quá trình Boot-up của STM32 (Bootup Process)

Khi cấp điện vào cho STM32, phần cứng sẽ tạo ra một tín hiệu **Reset Interrupt** gửi lên NVIC. Quá trình từ lúc có tín hiệu Reset cho đến khi chạy được vào hàm `main()` diễn ra như sau:

Yêu cầu 3: Biến `uwTick` được lưu ở đâu? Và có risk nào cho biến này không?

1. Biến `uwTick` được lưu ở đâu?

Trong bộ thư viện HAL, `uwTick` là một biến toàn cục (global variable) kiểu `__IO uint32_t` (`volatile uint32_t`), dùng để đếm số tick của hệ thống (thường tăng 1 đơn vị sau mỗi 1ms thông qua ngắt SysTick).

Bởi vì nó là biến toàn cục và được khởi tạo bằng 0 (`__IO uint32_t uwTick = 0;`), trình biên dịch sẽ xếp nó vào vùng nhớ `.bss` nằm trên **SRAM**.

Khi bạn mở file `.map` và search từ khóa `uwTick`, bạn sẽ thấy nó được cấp phát một địa chỉ nằm trong dải RAM (ví dụ `0x2000xxxx`).

2. Rủi ro (Risks) kinh điển khi sử dụng `uwTick` ?

